Reporte de Auditoría de Rendimiento

Aplicación médica de Realidad Mixta para Meta Quest 3

Campo	Valor
Proyecto	Proyecto_MetaQuest3 (rama `claude/inspiring-lamport-nqjcn9`, commit 5b221730)
Fecha	16/10/2026
Motor y paquetes (leídos de ProjectVersion.txt y Packages/manifest.json)	Unity 2022.3.62f1 · URP 14.0.12 · XR Interaction Toolkit 2.6.5 · OpenXR 1.14.3 · Meta XR SDK 201.0.0 · XR Hands 1.4.3 · KlakNDI 2.1.6
Plataforma objetivo	Android ARM64, IL2CPP, Vulkan (GLES3 como respaldo), GC incremental activo
Modo estéreo	OpenXR `m_renderMode: 1` (Single Pass Instanced; en Android se ejecuta como Multiview)
Tipo de revisión	Análisis estático pasivo. No se modificó ningún archivo del proyecto; el único archivo creado es este PDF.
Alcance	34 scripts C# de runtime (Assets/Scripts, Assets/ClippingPlane, MedicalMenuActions.cs), 7 shaders propios, escena de build, materiales, ajustes URP/OpenXR/Player.

Restricción de referencia: 8 GB de RAM compartida entre CPU y GPU en Meta Quest 3, de los que la aplicación solo dispone de una parte (el sistema operativo y el compositor reservan el resto). Las cifras de memoria de este informe se calculan a partir del código; los tiempos de CPU/GPU son estimaciones razonadas y deben confirmarse con Unity Profiler, Memory Profiler y OVR Metrics Tool en el dispositivo.

0. Aclaraciones sobre el contexto del proyecto

Antes de los hallazgos conviene corregir cuatro supuestos del encargo que no coinciden con el código actual, porque cambian dónde está el riesgo real:

- **No existe** `ClippingPlane.shader`**.** El shader de corte es `Assets/ClippingPlane/Shaders/ClippingPlane_URP.shader` (`Shader "APOSE/URP_ClippingPlane"`), un port a URP del original de Ronja Böhringer. El original en *surface shader* ya no está en el proyecto.
- **El shader de clipping no lee máscaras PNG.** El fondo negro de la tomografía se elimina en `Assets/Shaders/VolumeSlice.shader` (línea 122) descartando los vóxeles por debajo de `_AirThreshold = -300 HU`. Las máscaras PNG existen, pero se usan como capa UI (`RawImage overlay`) en `CTPlaneView.cs` (líneas 185-220).
- **fo-dicom no se ejecuta en la build.** La única escena en `EditorBuildSettings` es `SampleScene.unity`. `DicomVolumeViewer.cs` (fo-dicom) solo está en `DICOM_Test.unity`. En la build, el volumen 3D sale de dos `Texture3D` R8 de 512×512×267 precalculadas en el editor (`MedicalVolumeSetup.cs`), y los cortes 2D son JPG/PNG en `StreamingAssets` cargados con `UnityWebRequestTexture`.
- **Los 4 modelos anatómicos** (hígado, estómago, páncreas, vesícula; además existe `Mat_Clip_Heart.mat`) usan `APOSE/URP_ClippingPlane` con un segundo material `Mat_HoloOutline` añadido por las herramientas de editor.

El análisis de `DicomVolumeViewer.cs` se incluye igualmente (sección 1.B), porque es el que fallaría primero si esa ruta de carga se lleva a la build.

Resumen ejecutivo

Estado general: el proyecto ya corrige varios errores clásicos (texturas liberadas en `OnDestroy`, buffers reutilizados, Texture3D R8 no legible y sin mipmaps, SRP Batcher activo, GC incremental, Vulkan, MSAA 4x sin HDR). Los riesgos que quedan se concentran en cuatro puntos: (1) el coste GPU del raymarching volumétrico; (2) las cargas asíncronas de texturas que se cancelan sin liberarse al mover los sliders; (3) el uso generalizado de `MaterialPropertyBlock`, que desactiva el SRP Batcher en todos los órganos; y (4) tres shaders sin macros estéreo, que se ven en un solo ojo en Quest Link/PC.

No se ha encontrado ningún shader propio que vaya a salir rosado en la build: todos tienen un pase compatible con URP y llegan a la build referenciados desde materiales. En la sección 2.3 se detallan los riesgos residuales.

ID	Sev.	Área	Archivo:línea	Hallazgo
S1	**Alta**	GPU	VolumeRender.shader:195-263	Raymarching de 128 pasos con hasta 8 lecturas 3D por paso y transformación de matriz por paso.
R1	**Alta**	RAM/CPU	CTPlaneView.cs:160-161, 193-194	StopCoroutine en mitad de `using(UnityWebRequest)`: la petición no se aborta ni se libera; las texturas nativas quedan pendientes del finalizador del GC.
R7	**Alta**	RAM	DicomVolumeViewer.cs:105-144, 220-235	Pico de memoria gestionada de 280 MB o más (267 datasets y el volumen a la vez); solo en DICOM_Test.
U1	**Alta**	URP	ClippingPlaneController.cs:54-57 y otros 7	MaterialPropertyBlock en todos los órganos, volumen y paneles: esos renderers salen del SRP Batcher.
SH1	**Media**	Estéreo	HoloOutline / HoloGlassPanel / HoloPlatform	Faltan las macros de instancing/estéreo: se ven en un solo ojo en Single Pass Instanced en PC (Quest Link).
S2	**Media**	GPU	ClippingPlane_URP.shader:18, 22, 92	clip() en cola Geometry y Cull Off permanente.
S5	**Media**	GPU	HoloOutline.shader:62	El contorno se dibuja siempre, aunque alfa sea 0, y no respeta el plano de corte.
R2/R3	**Media**	RAM	CTPlaneView.cs:201, 226	Texturas JPG/PNG legibles (copia en CPU) en RGBA32 para imágenes en escala de grises.
R5	**Media**	GC	OrganInfoPanelController.cs:105	GetComponentsInChildren en LateUpdate: un array nuevo en cada frame.
U2	**Media**	CPU	ClippingPlaneController, VolumeCutSlice, TriPlaneView, CTMeasure2D	Escrituras por frame sin detectar cambios; reconstrucción de Canvas en cada frame.

ID	Sev.	Área	Archivo:línea	Hallazgo
U3	**Media**	XRI	SampleScene.unity	5 rayos con interacción UI contra 9 canvas y 221 Graphic con Raycast Target activo.
U4	**Media**	URP	URP-Balanced.asset	Sombras suaves de luz principal a 50 m y luces adicionales por píxel en Android.
S3/S4	**Baja**	Shader	ClippingPlane_URP.shader:33-34	Variantes de sombra que nunca se usan; faltan los pases ShadowCaster y DepthOnly.
S6	**Baja**	Shader	VolumeSlice.shader:118, 130	Muestreo con derivadas implícitas después de un discard no uniforme.
R10-R12	**Baja**	Datos/APK	StreamingAssets, Player	Densidad cuantizada a 8 bits, 71 MB de DICOM no usados en el APK, trazas de pila en Debug.Log.

1. Gestión de RAM y Garbage Collector (C#)

1.A Ruta de la build (SampleScene)

R1 · Alta — Cargas de corte canceladas sin Abort/Dispose al arrastrar el slider

Ubicación: `Assets/Scripts/CTPlaneView.cs` líneas 160-161 (`StopCoroutine(_loading)`), 193-194 (overlay), 197-220 y 222-254 (`using (UnityWebRequest request = ...)`). El mismo patrón aparece en `CTSliceViewer.cs:129-130` (no está en la escena de build).

Problema: Cada valor nuevo del slider para la corrutina anterior con `StopCoroutine`. Unity no ejecuta el `finally` implícito del `using` al detener una corrutina, así que la `UnityWebRequest` en vuelo ni se aborta ni se libera. La descarga y la decodificación del JPG siguen en segundo plano, y la textura que crea `DownloadHandlerTexture` (nunca recogida con `GetContent`) vive en memoria nativa hasta que el finalizador del GC libere el objeto.

Impacto en Quest 3: Un arrastre de 1-2 s a 72-120 Hz genera decenas de peticiones. Con `SliceSync` (vista Segmentación), cada tick lanza hasta 4 cargas: imagen y máscara en los dos visores sincronizados. Con 512×512 RGBA32 (1 MB en GPU, más 1 MB en CPU por ser legible, ver R2), un arrastre puede dejar temporalmente decenas de MB de texturas huérfanas y trabajo de decodificación inútil. Al soltar el slider el consumo baja, pero esos picos son los que provocan un cierre por falta de memoria (OOM) cuando coinciden con el volumen 3D (~134 MB) y el NDI.

Recomendación: Guardar la petición activa en un campo y, antes de lanzar otra, llamar a `_request.Abort(); _request.Dispose();`. Mejor aún, limitar la frecuencia: una sola petición en vuelo y, al terminar, cargar solo el último índice pedido, o cargar al soltar el slider. La alternativa más eficiente es quitar los JPG de la ruta VR: los tres visores 2D pueden muestrear la `Texture3D` que ya está en GPU con un shader como `VolumeSlice` (en un RenderTexture pequeño o en un quad). Así no hay descargas, ni decodificación, ni asignaciones por tick.

```
// Patrón sugerido (ilustrativo, NO aplicado al proyecto)
UnityWebRequest _req;
void Cancel() { if (_req != null) { _req.Abort(); _req.Dispose(); _req = null; } }
IEnumerator LoadSlice(string url) {
    Cancel();
    _req = UnityWebRequestTexture.GetTexture(url, nonReadable: true);
    var r = _req; yield return r.SendWebRequest();
    if (r != _req) yield break;               // ya se pidió otro corte
    ... DownloadHandlerTexture.GetContent(r) ...; Cancel();
}
```

R2 · Media — Texturas descargadas legibles: doble copia CPU+GPU

Ubicación: `CTPlaneView.cs:201` y `:226`; `CTSliceViewer.cs:138`.

Problema: `UnityWebRequestTexture.GetTexture(url)` usa `nonReadable = false` por defecto, así que la textura conserva una copia de los píxeles en memoria de CPU, además de la de GPU.

Impacto en Quest 3: En Quest la memoria es unificada, de modo que la copia legible ocupa RAM real: unos 1 MB extra por corte de 512×512 y hasta 6 texturas vivas (3 planos × imagen + máscara), más las huérfanas de R1.

Recomendación: Usar `UnityWebRequestTexture.GetTexture(url, true)`. Ningún script lee los píxeles de esas texturas: `CTMeasure2D` solo usa `texture.width/height`.

R3 · Media — Imágenes en escala de grises almacenadas como RGBA32

Ubicación: Cortes JPG y máscaras PNG en `CTPlaneView`; `DicomVolumeViewer.cs:152-157` (`TextureFormat.RGBA32`).

Problema: Una imagen de TC es monocanal. Guardarla en RGBA32 ocupa 4 veces lo necesario y cuadruplica el ancho de banda de muestreo y de subida a GPU.

Impacto en Quest 3: Es moderado en memoria absoluta (pocas texturas vivas), pero se suma a R1 y R2 en cada tick del slider.

Recomendación: Si se mantienen las imágenes, usar `R8` o `Alpha8` para los cortes generados en CPU, y ETC2/ASTC si se preimportan. La solución de fondo es la misma que en R1: muestrear la Texture3D.

R4 · Media — Asignaciones gestionadas en cada tick del slider

Ubicación: `CTPlaneView.cs:155` (`counter.text = $"{_slice + 1} / {sliceCount}"`), `:157-158` (interpolación y `Path.Combine`), `:190-191`, `:199`, `:224` (`"file://" + path`), objetos de corrutina y de `UnityWebRequest`.

Problema: Cada valor del slider crea varias cadenas, el enumerador de dos corrutinas y dos peticiones, y obliga a TextMeshPro a regenerar la malla del contador.

Impacto en Quest 3: Son unos pocos KB por tick. Con el GC incremental activo (`gcIncremental: 1`) no deberían verse pausas largas, pero esa basura constante adelanta las colecciones mientras se arrastra, que es justo cuando importa la fluidez.

Recomendación: Precalcular un `string[]` de rutas por plano al cambiar de órgano y usar `counter.SetText("{0} / {1}", _slice + 1, sliceCount)` (TMP sin asignaciones). Con la propuesta de R1 basada en Texture3D, este punto desaparece.

R5 · Media — GetComponentsInChildren en cada frame

Ubicación: `OrganInfoPanelController.cs:69-73` (`LateUpdate → PlaceNear`) → `:105` `target.GetComponentsInChildren<Renderer>()`.

Problema: Mientras el usuario apunta a un órgano se crea un array nuevo en cada frame para calcular los bounds.

Impacto en Quest 3: Basura continua a 72-120 Hz durante toda la interacción con los órganos, que es el uso principal de la app.

Recomendación: Cachear los renderers al llamar a `Show(target, ...)` o usar la sobrecarga sin asignación `GetComponentsInChildren(List<Renderer>)` con una lista reutilizada.

R6 · Baja — Asignaciones puntuales en InteractorModeSwitch y HoloHoverGlow

Ubicación: `InteractorModeSwitch.cs:181` (`GetComponentsInChildren<IXRSelectInteractor>()`); `HoloHoverGlow.cs:56-61`.

Problema: (a) Se crea un array cada frame en que los mandos están en uso en modo manos. (b) `Mathf.Lerp` hacia 0 converge de forma asintótica y `Mathf.Approximately(_current, 0)` solo se cumple cuando el valor cae por debajo de unos 1e-44. Por eso, tras salir del hover, el botón sigue escribiendo su MaterialPropertyBlock durante cientos de frames (estimado: varios segundos).

Impacto en Quest 3: Bajo, pero se multiplica por cada botón del menú que haya recibido hover.

Recomendación: (a) Cachear la lista. (b) Usar `Mathf.MoveTowards`, como ya hace `OrganSelectionInfo`, o forzar `_current = _target` cuando la diferencia baje de 1e-3.

Puntos correctos (no requieren acción)

- Volumen 3D: `Texture3D(…, TextureFormat.R8, false)` + `Apply(false, true)` (no legible, sin mipmaps): 67 MB cada una, unos 134 MB de GPU para densidad y órganos. Filtrado bilinear en densidad y point en etiquetas, como corresponde (MedicalVolumeSetup.cs:245-251).
- Liberación de texturas en `OnDestroy` (CTPlaneView, CTSliceViewer, JpgSliceViewer, DicomVolumeViewer) y destrucción de la textura anterior al recibir la nueva.
- Medidas 3D/2D: se instancian solo al pulsar, no por frame, y se destruyen con `Clear()`.

1.B Ruta fo-dicom (DICOM_Test.unity, fuera de la build actual)

R7 · Alta — Pico de memoria al cargar los 267 cortes con fo-dicom

Ubicación: `Assets/Scripts/DICOM/DicomVolumeViewer.cs` líneas 105 (`List<Slice> datasets`), 124-126 (`OrderBy(...).ToList()`), 136 (`new short[voxels]`), 220-235 (`request.downloadHandler.data` + `DicomFile.Open(stream)`), 271 (`GetFrame(0).Data`).

Problema: Se conservan los 267 `DicomDataset` completos (con PixelData de unos 512 KB cada uno, ≈140 MB) y después se reserva el volumen `short[]` (512×512×267×2 B ≈ 140 MB) mientras los datasets siguen vivos hasta el final de la corrutina. A eso se suman las copias de `downloadHandler.data` y

de `GetFrame(0).Data` por corte.

Impacto en Quest 3: El pico de heap gestionado supera los 280 MB. Con IL2CPP, el recolector (Boehm, no compactante) rara vez devuelve memoria al sistema, así que ese pico se convierte en huella permanente del proceso. En Quest, sumado a los ~134 MB del volumen en GPU, el NDI y la UI, compromete el presupuesto de RAM de la app.

Recomendación: No llevar fo-dicom a la ruta VR. Preconvertir la serie en el editor (como ya se hace con `ct_densidad_*.bytes.gz`) a un único binario R16 o directamente a Texture3D. Si hubiera que cargar DICOM en el visor: leer primero solo las cabeceras (posición del paciente) para ordenar, y convertir y descartar cada dataset en cuanto se lee, sin guardar la lista completa.

R8 · Media — Conversión y regeneración de cortes en el hilo principal

Ubicación: `DicomVolumeViewer.cs:291-296` (bucle por píxel con `double` y `Mathf.Clamp`, 16 cortes por frame), `:329-394` (`ShowAxial/Coronal/Sagittal` con `Gray()` → `Mathf.InverseLerp` por píxel, `SetPixels32` + `Apply` en cada valor del slider).

Problema: Cada frame de carga procesa unos 4,2 M de píxeles en el hilo principal. Cada movimiento del slider recorre 262 144 píxeles (axial) y sube 1 MB a GPU. El sagital recorre la memoria con salto de `_width` (fallos de caché).

Impacto en Quest 3: Estimado (sin medir): frames de decenas de ms durante la carga y caídas de frame al arrastrar el slider.

Recomendación: Tabla de consulta (LUT) precalculada de HU a byte, `Unity.Jobs` + Burst, texturas `R8`, o, preferiblemente, muestrear la Texture3D en GPU. Fuera de alcance: `slider.onValueChanged.RemoveAllListeners()` (línea 318) borra también los listeners de otros componentes.

1.C Datos y empaquetado

R10 · Baja — Densidad cuantizada a 8 bits

Ubicación: `MedicalVolumeSetup.cs:245` (`TextureFormat.R8`); `VolumeRender.shader:244` / `VolumeSlice.shader:119` (`density * 2000 - 1000`).

Problema: 2000 HU repartidos en 256 niveles dan ≈7,8 HU por nivel. Con la ventana de partes blandas (400 HU) solo quedan unos 51 grises distintos.

Impacto en Quest 3: Bandas visibles en tejido blando. Es un compromiso clínico, no de rendimiento.

Recomendación: Decisión del equipo: R16 o RHalf duplica la textura de densidad (+67 MB). Alternativa: guardar en 8 bits solo el rango útil, por ejemplo de -200 a +400 HU.

R11 · Baja — Peso del APK: StreamingAssets completos

Ubicación: `Assets/StreamingAssets`: 280 MB y 28 474 archivos (CT 209 MB, DICOM 71 MB).

Problema: StreamingAssets se copia entero al APK. La serie DICOM solo la usa `DICOM_Test.unity`, que no está en la build. Además, `Resources/MedicalLab360.png` (2048 px) no tiene referencias en los scripts ni en la escena revisados y se empaqueta por estar en Resources.

Impacto en Quest 3: Más tiempo de instalación y de E/S. Miles de archivos pequeños dentro del APK comprimido hacen que cada carga sea lenta.

Recomendación: Excluir la carpeta DICOM de las builds de Quest y empaquetar los cortes en un único binario o atlas por plano, o eliminarlos si se adopta la propuesta de Texture3D.

R12 · Baja — Trazas de pila en Debug.Log de release

Ubicación: `ProjectSettings.asset:57` `m_StackTraceTypes` = ScriptOnly para todos los tipos, Log incluido.

Problema: Cada `Debug.Log` captura su traza de pila (InteractorModeSwitch:201, NdiScreenStatus:120, WorkspaceRecenter, MedicalMenuActions...).

Impacto en Quest 3: Bajo: se llama solo ante cambios de estado.

Recomendación: Poner Log y Warning en None en las builds de release, o envolver los logs en `[Conditional("DEVELOPMENT_BUILD")]`.

2. Shaders: Single Pass Instanced / Multiview

2.1 Matriz de macros por shader

Leyenda: ✔ presente (con línea), ✘ ausente, n/a no aplica. Los números de línea son del propio archivo .shader.

Shader	multi_compile_ instancing	INPUT_ INSTANCE_ID (Attributes)	OUTPUT_ STEREO (Varyings)	SETUP_ INSTANCE_ID (vert)	INITIALIZE_ VERTEX_ OUTPUT_ STEREO	SETUP_STEREO_ EYE_INDEX_ POST_VERTEX (frag)	Veredicto
ClippingPlane_URP	✔ 32	✔ 55	✔ 65	✔ 72	✔ 74	✘ (no necesaria hoy)	**OK**
VolumeRender	✔ 55	✔ 85	✔ 98	✔ 105	✔ 107	✔ 180	**OK**
VolumeSlice	✔ 51	✔ 77	✔ 85	✔ 92	✔ 94	✘ (no necesaria hoy)	**OK**
HoloOutline	✘ (29-31)	✘ (33-37)	✘ (39-42)	✘ (50-58)	✘ (50-58)	n/a	**Media**
HoloGlassPanel	✘ (31-33)	✘ (35-39)	✘ (41-46)	✘ (58-66)	✘ (58-66)	n/a*	**Media**
HoloPlatform	✘ (28-30)	✘ (32-36)	✘ (38-42)	✘ (51-57)	✘ (51-57)	n/a	**Media**
UIRoundedRect (UI)	n/a	✔ 69	✔ 79	✔ 92	✔ 93	n/a	**OK**

* HoloGlassPanel calcula `viewDirWS` en el vertex shader (línea 64), donde el índice de ojo es válido. Si ese cálculo se moviera al fragment shader, haría falta `UNITY_SETUP_STEREO_EYE_INDEX_POST_VERTEX`.

2.2 Diagnóstico del renderizado en un solo ojo

SH1 · Media — HoloOutline, HoloGlassPanel y HoloPlatform sin soporte de instancing estéreo

Ubicación: `Assets/Shaders/HoloOutline.shader` (líneas 29-58), `HoloGlassPanel.shader` (31-66), `HoloPlatform.shader` (28-57).

Problema: Les faltan `#pragma multi_compile_instancing`, `UNITY_VERTEX_INPUT_INSTANCE_ID`, `UNITY_VERTEX_OUTPUT_STEREO`, `UNITY_SETUP_INSTANCE_ID` y `UNITY_INITIALIZE_VERTEX_OUTPUT_STEREO`. Además, `Varyings OUT;` se declara sin inicializar (HoloOutline:52, HoloGlassPanel:60, HoloPlatform:53).

Impacto en Quest 3: Depende de la ruta de renderizado. **(a) Quest nativo (APK Android, Multiview Vulkan):** según la documentación de Unity para Android, los shaders con vertex shader propio solo necesitan código extra para usar `unity_StereoEyeIndex`. La matriz de vista de cada ojo se resuelve en el vértice, así que lo más probable es que se vean en ambos ojos. **(b) Quest Link / Play Mode en PC** (Standalone también tiene `m_renderMode: 1`, es decir, Single Pass Instanced sobre D3D11): sin estas macros el GPU no sabe en qué capa del texture array escribir, y el objeto aparece solo en el ojo izquierdo. Es el síntoma clásico. Los paneles del menú, las peanas y el contorno de selección fallarían así durante las pruebas con Link.

Recomendación: Añadir en los tres shaders lo siguiente (ejemplo con HoloOutline; mismo patrón en los otros dos). Verificarlo con el shader de depuración `XR/StereoEyeIndexColor` de Unity, que pinta cada ojo de un color, tanto en Link como en el APK.

```
HLSLPROGRAM
#pragma vertex vert
#pragma fragment frag
#pragma multi_compile_instancing                        // <- añadir (línea ~30)
#include "Packages/com.unity.render-pipelines.universal/ShaderLibrary/Core.hlsl"

struct Attributes {
    float4 positionOS : POSITION;
    float3 normalOS   : NORMAL;
    UNITY_VERTEX_INPUT_INSTANCE_ID                      // <- añadir (línea ~37)
};
struct Varyings {
    float4 positionHCS : SV_POSITION;
    UNITY_VERTEX_OUTPUT_STEREO                          // <- añadir (línea ~42)
};
```

```
Varyings vert(Attributes IN) {
    Varyings OUT = (Varyings)0;                          // <- inicializar (línea 52)
    UNITY_SETUP_INSTANCE_ID(IN);                         // <- añadir
    UNITY_INITIALIZE_VERTEX_OUTPUT_STEREO(OUT);          // <- añadir
    ...
}
half4 frag(Varyings IN) : SV_Target {
    UNITY_SETUP_STEREO_EYE_INDEX_POST_VERTEX(IN);        // solo si frag usa datos de cámara/ojo
    ...
}
```

ClippingPlane_URP, VolumeSlice y UIRoundedRect

Tienen todas las macros necesarias. A ClippingPlane_URP (frag, línea 87) y VolumeSlice (frag, línea 109) les falta `UNITY_SETUP_STEREO_EYE_INDEX_POST_VERTEX(IN)`, pero hoy no es un error: sus fragment shaders no leen nada que dependa del ojo (posición de cámara, matrices de vista, `GetWorldSpaceViewDir`). Se recomienda añadirla de forma preventiva, porque si en el futuro se añade iluminación especular, rim, niebla o cualquier uso de `_WorldSpaceCameraPos` en el fragment, el ojo derecho usaría la cámara del izquierdo. Es exactamente el fallo que ya se corrigió en VolumeRender (commit 17fc0e0f, comentario de las líneas 112-123 del shader, que ahora pasa `cameraOS` desde el vertex). UIRoundedRect sigue el mismo patrón que `UI/Default` de Unity y es correcto.

2.3 Diagnóstico del «shader rosado»

En URP, el color magenta aparece cuando un shader no compila para la plataforma o no tiene ningún pase que URP sepa dibujar (el caso del `#pragma surface` original). Lo revisado:

- Los 7 shaders propios son HLSL/CG sin `#pragma surface`. Los de URP declaran `"RenderPipeline"="UniversalPipeline"` con `LightMode="UniversalForward"`; UIRoundedRect no tiene LightMode y URP lo dibuja como SRPDefaultUnlit. **Ninguno tiene causa de magenta.**
- `Shader.Find` solo aparece en scripts de `Assets/Editor`. En runtime los shaders llegan a la build porque los referencian materiales de la escena, así que no hay riesgo de que falten en la build por depender de Shader.Find.
- VolumeRender y VolumeSlice usan `#pragma target 3.5` y texturas 3D, soportadas por Vulkan y GLES 3.x del Quest.
- Riesgo residual (no verificable sin la carpeta Library): `Assets/Mat_UI_Menu.mat` apunta a un shader con GUID `933532a4fcc9baf4fa0491de14d08ed7` que no está en Assets (posiblemente un paquete). No está referenciado en SampleScene, así que hoy no afecta a la build.
- Si aparece magenta solo en el visor, revisar en este orden: el log de compilación de shaders de la build Android (Vulkan), que los materiales no se hayan reasignado a `Standard` (Built-in), y el stripping de variantes de URP (ver S3).

2.4 Otros problemas de rendimiento en shaders

S1 · Alta — VolumeRender: coste del raymarching por píxel

Ubicación: `Assets/Shaders/VolumeRender.shader`: `_Steps = 128` (línea 21), bucle 211-263, `TransformObjectToWorld` por paso (215), 2 lecturas 3D por paso (223-224), `SurfaceNormal` con 6 lecturas 3D más y `Lighting` con 2 transformaciones y 2 `normalize` (139-175, llamado en 254).

Problema: En el peor caso, cada píxel cubierto por el cubo hace 128 × (2 + 6) = 1024 lecturas de textura 3D y 128 multiplicaciones de matriz, todo en `float`. El plano de corte se evalúa en espacio mundo dentro del bucle, aunque es constante. La terminación temprana (`accumulated.a > 0.98`) ayuda poco con `_TissueOpacity = 0.10`, porque la mayoría de los rayos atraviesan el volumen entero.

Impacto en Quest 3: Es, con diferencia, el mayor consumidor de GPU de la escena. Escala con el área de pantalla que ocupa el cubo en cada ojo: acercarse al volumen o meter la cabeza dentro (`Cull Front`) lo multiplica. Riesgo de bajar de 72/90 Hz y de que el sistema active la reproyección.

Recomendación: (1) Calcular la intersección rayo-plano de forma analítica y recortar `tmin/tmax` antes del bucle. El plano puede pasarse ya en espacio objeto desde C# (`worldToLocalMatrix`), lo que elimina las líneas 215-221 del bucle. (2) Precalcular el gradiente en una Texture3D de menor resolución (por ejemplo 256×256×134 RGBA8 ≈ 35 MB) o usar diferencias hacia delante (3 lecturas en vez de 6). (3) Pasos adaptativos: 48-64 mientras el usuario mueve el plano o el volumen, 128 en reposo. (4) Saltar el

espacio vacío con una textura de ocupación de baja resolución. (5) Usar `half` en el color y el acumulado. (6) Si aun así no llega al presupuesto, renderizar el volumen a media resolución en un RenderTexture y componerlo.

S2 · Media — ClippingPlane_URP: clip() en cola opaca y Cull Off permanente

Ubicación: `ClippingPlane_URP.shader:18` (`"Queue"="Geometry"`), `:22` (`Cull Off`), `:92` (`clip(-distanceToPlane)`).

Problema: Un `discard` en materiales opacos de la cola Geometry obliga a la GPU de tiles (Adreno 740) a desactivar optimizaciones de Z temprana en esos draws, y se dibujan intercalados con el resto de opacos. `Cull Off` rasteriza también las caras traseras de los 4-5 órganos aunque el corte esté desactivado, porque `_PlaneDistance = -100000` sigue ejecutando el clip.

Impacto en Quest 3: Hasta 2 veces el trabajo de fragmentos en los órganos y peor rechazo por profundidad.

Recomendación: Mover a `"Queue"="AlphaTest" "RenderType"="TransparentCutout"`. Con el corte desactivado, asignar un material o variante sin clip y con `Cull Back`: por ejemplo, un keyword `_CLIPPING_ON` con `#pragma shader_feature_local` y `Cull [_Cull]` gobernado desde `ClippingPlaneController.SetClippingEnabled`.

S3 · Baja — Variantes de sombra inútiles en ClippingPlane_URP

Ubicación: `ClippingPlane_URP.shader:33-34` y `:104`.

Problema: Se declaran `_MAIN_LIGHT_SHADOWS` y `_MAIN_LIGHT_SHADOWS_CASCADE` como dos `multi_compile` independientes (4 combinaciones × instancing), pero `GetMainLight()` se llama sin coordenada de sombra, así que nunca se muestrean sombras.

Impacto en Quest 3: Variantes extra en la build y en memoria de shaders sin ningún efecto visual.

Recomendación: Eliminar esas dos líneas. Si se quieren sombras, calcular `TransformWorldToShadowCoord` y llamar a `GetMainLight(shadowCoord)` con los keywords tal como los declara URP 14.

S4 · Baja — Sin pases ShadowCaster / DepthOnly / DepthNormals

Ubicación: `ClippingPlane_URP.shader` (un único pase, `FallBack Off`, línea 114).

Problema: Los órganos no proyectan sombras ni aparecen en la textura de profundidad.

Impacto en Quest 3: Hoy no tiene coste (`m_RequireDepthTexture: 0` y SSAO inactivo). Si se activan depth texture, depth priming o SSAO, los órganos desaparecerán de esos efectos o saldrán sin recortar.

Recomendación: Si se activan esas funciones, añadir un pase DepthOnly que aplique el mismo `clip`.

S5 · Media — HoloOutline siempre dibujado e ignorando el corte

Ubicación: `HoloOutline.shader:17, 24-26, 62`; asignado como material adicional a los órganos (`Editor/MedicalViewerTools.cs`, `MedicalOrganLibrary.cs:193-198`).

Problema: El casco extruido se dibuja en cada frame, con blending, aunque `_OutlineAlpha = 0`: devuelve alfa 0, pero rasteriza y mezcla igual. Tampoco lee `_PlaneNormal`/`_PlaneDistance`, así que al seleccionar un órgano cortado se ve el contorno de la silueta completa.

Impacto en Quest 3: Un draw transparente extra por órgano y ojo, con fill rate de toda la silueta, más un artefacto visual en la función principal (el corte).

Recomendación: Activar el material de contorno solo mientras `_OutlineAlpha > 0` (por ejemplo, cambiando `sharedMaterials` o activando un renderer hijo desde `OrganSelectionInfo`), y aplicar en su fragment el mismo `clip` del plano.

S6 · Baja — VolumeSlice: derivadas implícitas tras discard no uniforme

Ubicación: `VolumeSlice.shader:116` y `:122` (discard) → `:118` y `:130` (`SAMPLE_TEXTURE3D`).

Problema: Las lecturas con derivadas implícitas después de un `discard` que no es uniforme en el quad de 2×2 tienen gradientes indefinidos en algunos drivers. Las Texture3D no tienen mipmaps, así que esas derivadas no aportan nada.

Impacto en Quest 3: Posibles artefactos en los bordes del cuerpo según el driver, y un coste de cálculo de LOD innecesario.

Recomendación: Usar `SAMPLE_TEXTURE3D_LOD(..., 0)`, como ya hace VolumeRender. Relacionado: `TriPlaneView.cs:102` dimensiona los tres planos a la diagonal del volumen × 1,02, así que gran parte de cada quad se rasteriza para acabar descartada (sobredibujado con escritura de Z).

S7 · Baja — Shaders de UI y paneles holográficos

Ubicación: `UIRoundedRect.shader:134-135`; `HoloGlassPanel.shader` / `HoloPlatform.shader`.

Problema: UIRoundedRect ejecuta `UnityGet2DClipping` y `clip()` en todos los fragmentos, sin el keyword `UNITY_UI_CLIP_RECT`. Los paneles transparentes grandes suman sobredibujado y usan `float` donde basta `half`.

Impacto en Quest 3: Bajo de forma individual. Relevante por el número de paneles del menú.

Recomendación: Añadir `#pragma multi_compile_local _ UNITY_UI_CLIP_RECT` y condicionar el recorte. Usar `half` en color y alfa.

3. Cuellos de botella en URP y XR Interaction Toolkit

U1 · Alta — MaterialPropertyBlock desactiva el SRP Batcher en órganos, volumen y paneles

Ubicación: `ClippingPlaneController.cs:54-57` (todos los órganos, cada frame), `VolumeCutSlice.cs:55-88` (cada frame), `TriPlaneView.cs:107-109` (cada frame), `OrganSelectionInfo.cs:109-114`, `HoloHoverGlow.cs:59-61`, `MedicalMenuIntro.cs:110-114`, `VolumeDisplay.cs:74-80`, `NdiScreenStatus.cs:89-93`.

Problema: La documentación de Unity indica que `MaterialPropertyBlock` no es compatible con el SRP Batcher y que un renderer con MPB se sale de ese camino. URP-Balanced tiene `m_UseSRPBatcher: 1` y los shaders están preparados (CBUFFER UnityPerMaterial), pero en la práctica ningún objeto relevante lo aprovecha.

Impacto en Quest 3: Más coste de CPU (hilo de render) por draw call, en un dispositivo cuyo cuello de botella suele ser la CPU. El código lo usa precisamente pensando en ahorrar, pero en URP el efecto es el contrario.

Recomendación: (1) El plano de corte es el mismo para todos los objetos: quitar `_PlaneNormal` y `_PlaneDistance` del bloque Properties y del CBUFFER `UnityPerMaterial`, declararlos como uniforms globales y escribirlos una vez por frame con `Shader.SetGlobalVector/SetGlobalFloat` (solo si `planeHandle.hasChanged`). (2) Para hover, contorno y fundido: instancias de material propias por objeto, creadas una vez en `Awake` con `renderer.material` y destruidas en `OnDestroy`, que siguen siendo compatibles con el SRP Batcher. (3) `_VolumeWorldToObject` se puede escribir en el material del corte en vez de en un MPB. Validar con Frame Debugger ("SRP Batch") antes y después.

U2 · Media — Trabajo por frame sin detección de cambios

Ubicación: `ClippingPlaneController.cs:41-59`; `VolumeCutSlice.cs:47-100` (Get/Set del MPB dos veces y relectura en `IsCutting()`); `TriPlaneView.cs:76-110`; `CTMeasure2D.cs:87-112`.

Problema: Se reescriben propiedades y transforms en cada frame aunque nada cambie. En `CTMeasure2D`, asignar `sizeDelta` y `localRotation` a RectTransforms en `LateUpdate` marca el Canvas como sucio, y UGUI reconstruye el lote del canvas del visor en cada frame mientras haya medidas 2D. `VolumeCutSlice.IsCutting` deduce el estado leyendo el MPB que escribe otro script: es frágil y depende del orden de ejecución (`[DefaultExecutionOrder(100)]`).

Impacto en Quest 3: CPU del hilo principal y reconstrucción de Canvas constantes.

Recomendación: Usar `transform.hasChanged` o flags de suciedad, y exponer `IsClippingEnabled` como propiedad pública del controlador. En CTMeasure2D, recalcular solo al cambiar el tamaño del rect (`OnRectTransformDimensionsChange`) o la textura.

U3 · Media — Raycasting de UI de XRI sobredimensionado

Ubicación: `SampleScene.unity`: 5 `XRRayInteractor` con `m_EnableUIInteraction: 1`, 9 Canvas, 12 componentes de raycaster gráfico, 221 Graphic con `m_RaycastTarget: 1` (y 8 con 0) y `m_RaycastMask` sin filtrar.

Problema: Cada rayo activo hace cada frame raycast de física contra todas las capas y raycast de UI contra todos los canvas, probando cada Graphic con Raycast Target. Textos e imágenes decorativas también participan.

Impacto en Quest 3: CPU del hilo principal, que crece con el número de rayos activos (manos y mandos, aunque `InteractorModeSwitch` ya limita a un juego).

Recomendación: Desactivar Raycast Target en textos, iconos y fondos no interactivos. Limitar `Raycast Mask` a las capas Organos/UI. Quitar raycasters de los canvas que solo muestran información (panel de órgano, contador). Desactivar UI Interaction en los rayos que no la necesitan.

U4 · Media — Ajustes de URP para Android mejorables

Ubicación: `QualitySettings` → Android usa el nivel 1 "Balanced" → `Assets/Settings/URP-Balanced.asset`.

Problema: Sombras de luz principal activas con `m_SoftShadowsSupported: 1`, `m_ShadowDistance: 50` y mapa de 1024; `m_AdditionalLightsRenderingMode: 1` (por píxel); cámara con `m_RenderShadows: 1`. Los órganos (ClippingPlane_URP) no proyectan ni reciben sombras (S3/S4), así que el pase de sombras trabaja para el resto de la escena.

Impacto en Quest 3: Un pase de sombras completo y el filtrado suave por píxel en todos los materiales Lit, en una app de passthrough donde las sombras aportan poco.

Recomendación: Desactivar las sombras de la luz principal (o bajar la distancia a 2-3 m y quitar las sombras suaves), y poner las luces adicionales en Disabled o Per Vertex. Es correcto y conviene mantener: HDR desactivado, MSAA 4x, SRP Batcher, post-procesado desactivado, Vulkan, `optimizeBufferDiscards: 1` y foveated rendering activado como feature de OpenXR. No se encontró ningún script que fije el nivel de foveación en tiempo de ejecución: verificar en el dispositivo que realmente está activo.

U5 · Baja — Otras observaciones de interacción y streaming

Ubicación: `ClippingPlaneInput.cs:72`, `MeasureTool.cs:107-109`, `CTMeasure2D.cs:119-121`, `MenuToggleInput.cs`, `InteractorModeSwitch.cs` (OVRInput); `OVRHandXRController.cs:19` (`XRBaseController`); NdiReceiver (KlakNDI) en la escena.

Problema: Conviven dos pilas de entrada (XRI y OVRInput) consultadas en paralelo cada frame. `XRBaseController` está marcado como obsoleto en XRI 3.x. El receptor NDI decodifica vídeo en CPU y actualiza una textura de forma continua.

Impacto en Quest 3: Bajo en la entrada. El NDI puede ser significativo en CPU y memoria según la resolución del stream: no se puede cuantificar sin perfilar.

Recomendación: Perfilar el NDI con el Profiler en el dispositivo y limitar la resolución o los fps del emisor. Planificar la migración de `OVRHandXRController` a XR Hands / input actions al actualizar XRI.

Observación funcional (fuera del alcance de rendimiento)

VolumeRender usa `Cull Front` + `ZWrite Off` con ZTest por defecto. Si un objeto opaco queda dentro del cubo, la cara trasera del cubo falla el test de profundidad y se pierde el rayo entero en esos píxeles, incluido el tramo de volumen que hay delante del objeto. Se menciona sin evaluar a fondo.

4. Plan de acción priorizado

#	Acción	Hallazgos	Beneficio esperado
1	Abortar/liberar las UnityWebRequest al cambiar de corte, cargar en modo «último pedido gana» y pasar `nonReadable: true`.	R1, R2, R4	Elimina picos de memoria y trabajo inútil al arrastrar sliders.
2	Optimizar VolumeRender: plano analítico fuera del bucle, gradiente precalculado o más barato, pasos adaptativos.	S1	Mayor margen de GPU; es el principal riesgo de bajar de la tasa de refresco.
3	Sustituir los MPB del plano de corte por globals y los de hover/fundido por instancias de material.	U1, U2	Recupera el SRP Batcher y reduce la CPU de render.
4	Añadir las macros estéreo a HoloOutline, HoloGlassPanel y HoloPlatform y validar con XR/StereoEyeIndexColor.	SH1	Visualización correcta en ambos ojos en Quest Link/PC.
5	Clipping: cola AlphaTest, variante sin clip y con Cull Back cuando el corte está apagado; contorno solo al seleccionar y recortado por el plano.	S2, S5	Menos fill rate en los órganos y contorno coherente con el corte.
6	Reducir el raycasting de UI (Raycast Target, máscaras, raycasters) y el trabajo por frame sin cambios.	U2, U3, R5	Menos CPU del hilo principal y menos basura para el GC.
7	Revisar las sombras y luces de URP-Balanced y confirmar la foveación en el dispositivo.	U4	Menos coste GPU fijo.
8	Mantener fo-dicom fuera de la ruta VR (preconversión en el editor) y excluir StreamingAssets/DICOM del APK.	R7, R8, R11	Evita un pico de más de 280 MB y reduce el APK en 71 MB.

5. Metodología, límites y fuentes

- Lectura completa de los shaders propios y de los scripts de runtime; cruce de GUID de scripts y materiales con `SampleScene.unity` para separar el código que entra en la build del que no; lectura de `OpenXR Package Settings.asset`, `QualitySettings`, `URP-Balanced*.asset` y `ProjectSettings.asset`.
- **Verificado en el código:** rutas, números de línea, formatos de textura, ajustes de URP/OpenXR/Player, conteos de la escena. **Calculado:** tamaños de memoria a partir de dimensiones y formatos. **Estimado (sin perfilar):** tiempos de CPU/GPU e impacto en fps.
- No se dispone de la carpeta `Library` ni de un Quest 3 en este entorno: no se ha compilado el proyecto ni medido en el dispositivo. Las prioridades deben confirmarse con Unity Profiler, Memory Profiler, Frame Debugger y OVR Metrics Tool.
- No se han modificado archivos `.cs`, `.shader`, `.prefab`, `.asset` ni ningún otro del proyecto. Los fragmentos de código de este informe son ilustrativos.

Fuentes consultadas para las afirmaciones sobre el comportamiento de Unity

- Single Pass Instancing y macros necesarias: docs.unity3d.com/6000.1/Documentation/Manual/SinglePassInstancing.html
- Multiview en Android (cuándo los shaders necesitan código extra): docs.unity3d.com/cn/2022.3/Manual/Android-SinglePassStereoRendering.html
- Incompatibilidad de MaterialPropertyBlock con el SRP Batcher: docs.unity.cn/Manual/SRPBatcher-Incompatible.html y referencia de la API MaterialPropertyBlock (docs.unity.com).
- Parámetro nonReadable (por defecto false) de UnityWebRequestTexture.GetTexture: docs.unity.cn/ScriptReference/Networking.UnityWebRequestTexture.GetTexture.html
- finally/Dispose no ejecutado al detener corrutinas: issue tracker de Unity («finally block of try/finally in coroutine is not called…», estado Postponed) e hilo forum.unity.com/threads/proper-way-to-stop-coroutine.704210.